

MedInsure Case Study: Operationalizing ML

Addressing key challenges in deploying machine learning models reliably in production environments.

Shreyas Bharadwaj 1RV23AI096

Shaurya Singh 1RV23AI092

Shubhaditya B 1RV23AI099

Shreya Mohan 1RV23AI095





The Peril of Environment Inconsistency

Minor discrepancies in development and production environments can lead to significant, unpredictable changes in model behavior. These inconsistencies are often subtle but can have critical impacts on model performance and reliability.

Why Python/Library Versions Alter Model Behavior

Numerical Differences

Updated algorithms in libraries like NumPy, Pandas, or Scikit-learn can subtly change how computations, data scaling, or randomization functions behave, leading to divergent results.

Serialization Conflicts

Models saved (pickled) under one Python version (e.g., 3.8) may load incorrectly or behave differently in another (e.g., 3.10) due to changes in serialization protocols or class definitions.

Preprocessing Variations

Differences in handling missing values, scaling techniques, or encoding methods across environments can cause inconsistent feature transformations, impacting model input.

Dependency Drift

Library updates can introduce new default behaviors, deprecate parameters, or alter internal logic, all of which modify how a model processes data and generates predictions.

“

Case Study Impact

Staging server used Python 3.10 while development used Python 3.8. Updated libraries altered hospital-type encoding and risk-score scaling. Decision tree behavior changed due to new splitting optimizations. Result ³ prediction drift, unstable approvals, and rejected claim requests.

”

Ensuring Environment Consistency for Production

To mitigate the risks of environment inconsistencies, proactive strategies must be implemented throughout the ML lifecycle.



Key Steps for Production Readiness

01

Freeze Dependencies

Generate and utilize `requirements.txt` or `environment.yml` with exact version numbers for all packages to lock down the environment.

03

Track Environment Metadata

Leverage tools like MLflow or DVC to store comprehensive metadata, including Python versions, dependency hashes, and training environment details, for reproducibility.

05

End-to-End Model Packaging

Bundle preprocessing logic and the trained model into a single, immutable artifact (e.g., Scikit-learn Pipeline, ONNX, TF SavedModel) for consistent deployment.

07

Schema Validation Tests

Add schema validation tests to detect missing features like `hospital_risk_score` and prevent ETL pipeline conflicts that could cause feature-mismatch errors in production.

02

Containerize the Environment

Use Docker or similar containerization tools to package the entire environment, guaranteeing identical versions across development, staging, and production.

04

Automated CI/CD Compatibility Tests

Integrate automated tests within your CI/CD pipeline to detect library or environment drift early in the development and deployment process.

06

Avoid System-Level Dependencies

Minimize reliance on external system-level libraries, ensuring core mathematical libraries and drivers remain consistent across all deployment environments.

The Imperative of Stress Testing

Before deploying any ML model to production, rigorous stress testing is crucial. It's the only way to uncover hidden vulnerabilities and ensure resilience under real-world loads.



Why Stress Testing Is Necessary

Identifies Bottlenecks

ML inference is computationally heavy and can slow down under real-world traffic. Production traffic (5,000 claims/min) is much higher than notebook testing, revealing bottlenecks in preprocessing, API routing, or database lookups.

Ensures Scalability

Verifies how the API behaves and scales under anticipated peak traffic, ensuring the model meets latency SLAs for customer-facing applications without degradation.

Prevents Failures

Uncovers critical issues like timeouts, memory leaks, thread exhaustion, or message queue overflows that can lead to service interruptions during peak claim processing periods.

Protects User Experience

Guarantees that the ML service remains responsive and avoids slowdowns in user-facing applications, preserving a smooth user experience.

Validates Real-World Readiness

Confirms the model and API's stability and reliability under sustained, production-level operational loads and unpredictable spikes.

Stabilizing the API Under Heavy Load

Designing for resilience is paramount. Implement robust architectural patterns and optimization techniques to ensure your ML API stands strong against traffic surges.



Methods for API Stability



Horizontal Scaling

Deploy multiple replicas of the ML service behind a load balancer. Utilize Kubernetes autoscaling to dynamically adjust the number of pods based on demand.



Asynchronous Processing

Implement message queues (e.g., Kafka, RabbitMQ) for request handling. The API accepts jobs, workers process them, and clients retrieve results later, smoothing traffic spikes.



Model Optimization

Convert models to optimized formats like ONNX or TensorRT for faster inference. Employ batch processing for high throughput and apply quantization to reduce latency.



Caching

Cache frequently repeated inference results. This significantly reduces the load on the ML model by preventing redundant computations for identical requests.



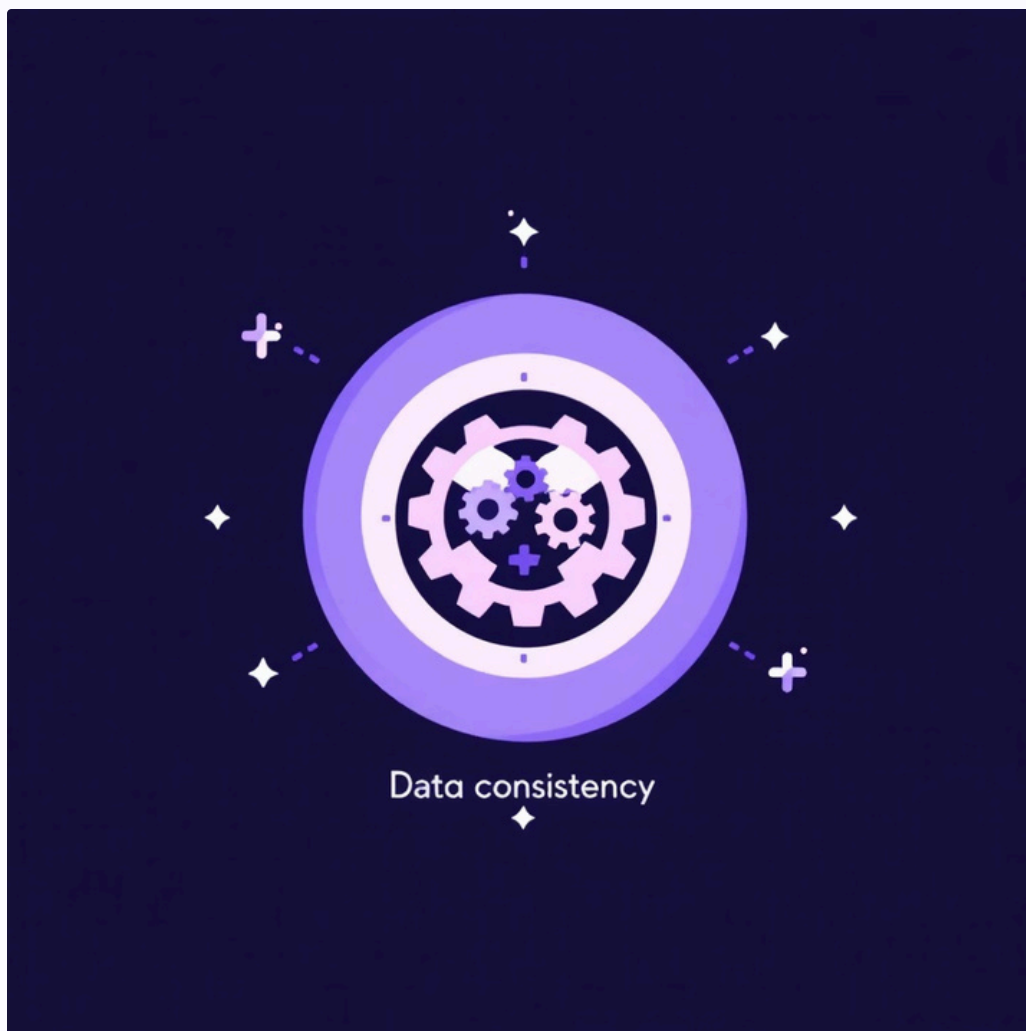
Rate Limiting & Backpressure

Protect the API from being overwhelmed by sudden bursts of requests. Implement rate limiting and backpressure mechanisms to ensure reliability and fair usage across all clients.

Key Takeaways for Production ML

Q1: Environment Consistency

- Version mismatches lead to inconsistent model results and prediction drift.
- Case study: Python 3.10 vs 3.8 caused hospital-type encoding changes and unstable claim approvals.
- Solutions: Frozen environments, Docker, ML metadata tracking, schema validation tests, and end-to-end pipeline packaging.
- Benefits: Prevents prediction drift, ensures auditor reproducibility, and supports reliable deployment of regulated health-insurance models.



Q4: Stress Testing & Stability

- Stress testing exposes performance and reliability issues under load.
- Solutions: Autoscaling, asynchronous queues, model optimization, caching, and robust rate control mechanisms.



By addressing these critical aspects, MedInsure can build and deploy highly reliable and performant